Kunming AI+

An Applied Introduction to AI with PyTorch

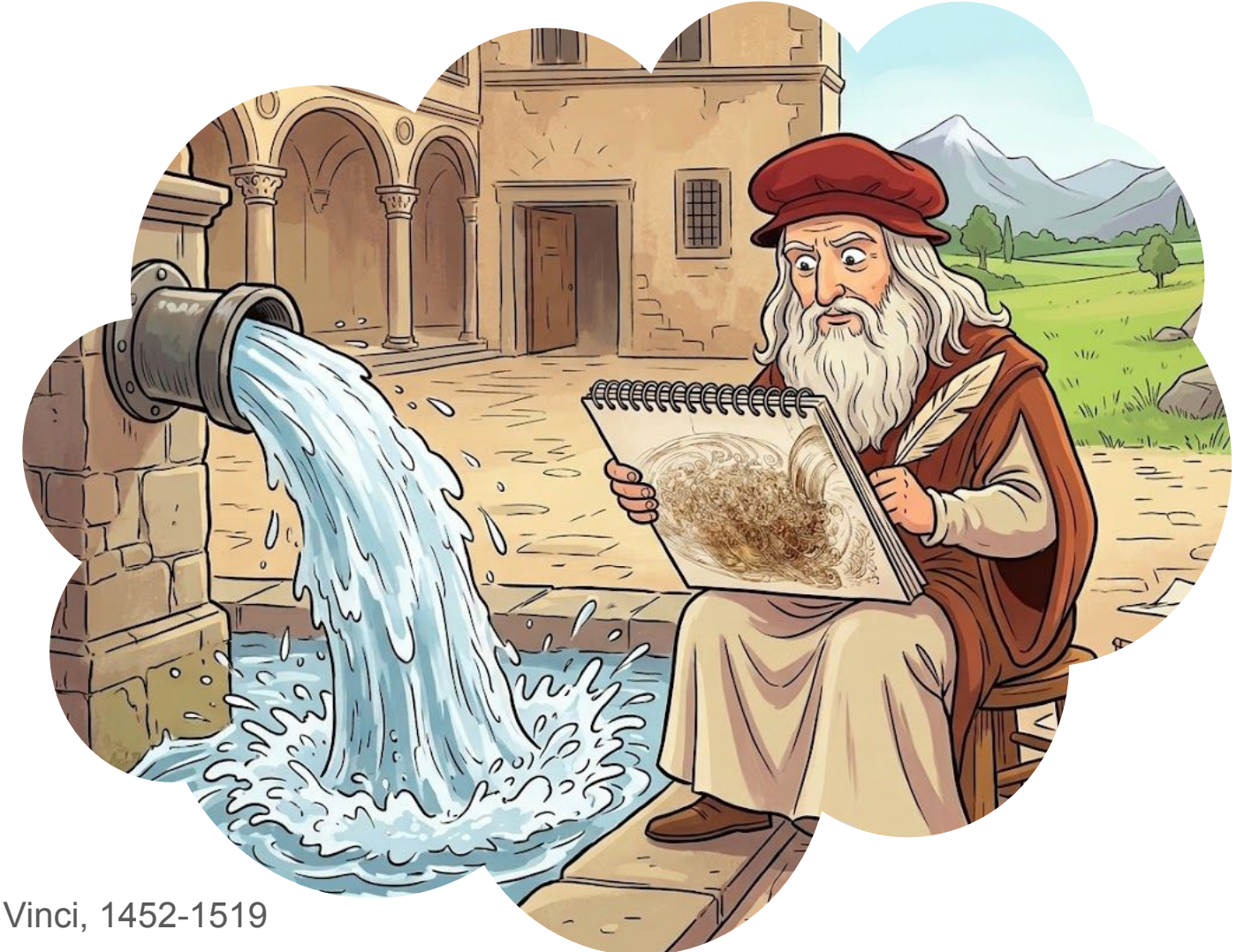
Luca Ghafourpour

UNIVERSITY OF CAMBRIDGE
Department of Engineering

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance + *Weights and Biases*
Practical Workshop 3	Initialisation and regularisation
Workshop 4	Convolutional neural networks
Practical Workshop 4	Dimensionality reduction, k-fold CV, confusion matrices
Academic Lecture 1	**An introduction to scientific machine learning**
Academic Lecture 2	Advanced methods in scientific machine learning
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

PART 1

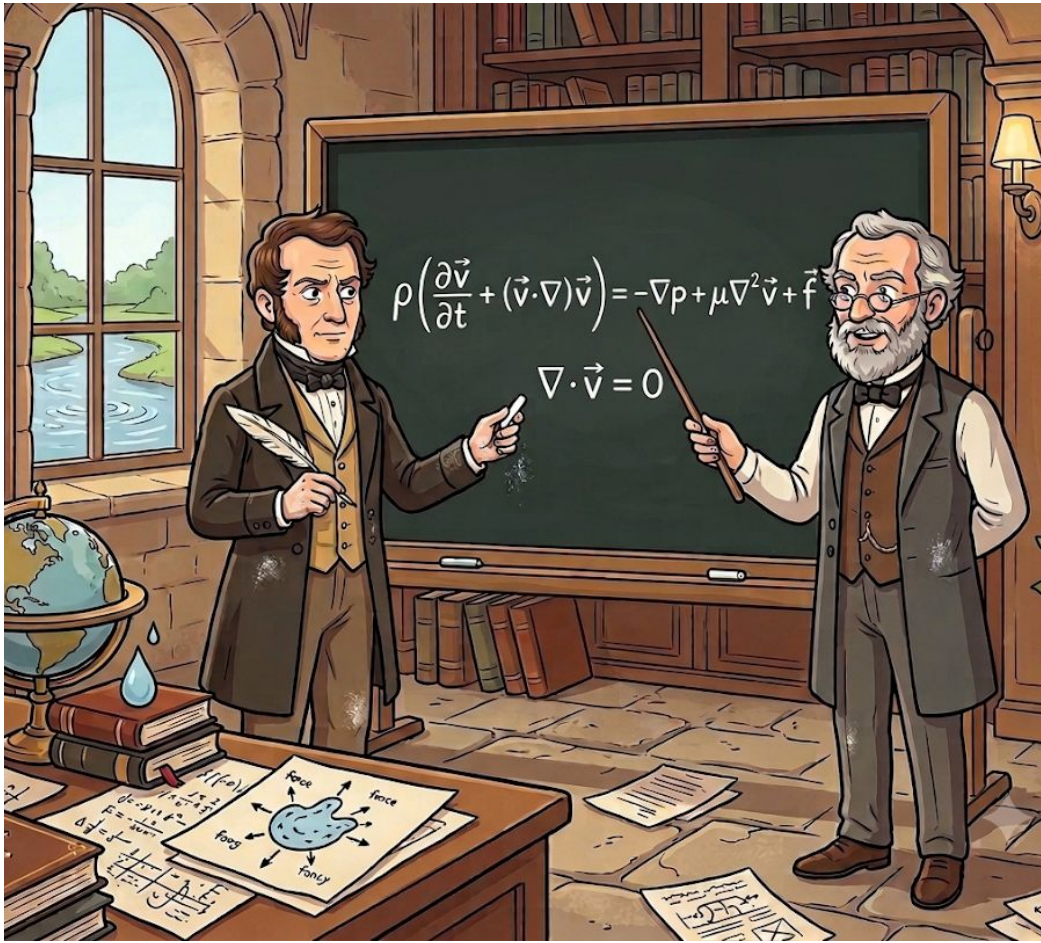
Classical Scientific Modelling



Leonardo da Vinci, 1452-1519

Da Vinci could see the structure
He just didn’t have the language to describe it

Using Isaac Newton's 2nd law of motion (1687), Louis Navier (1822) and George Gabriel Stokes (1842-1850) were able to mathematically describe what Da Vinci saw as he sat by the water.



The Navier-Stokes Equations

$$\underbrace{\rho\left(\underbrace{\frac{\partial \mathbf{v}}{\partial t}}_{\text{unsteady term}} + \underbrace{\mathbf{v}\cdot\nabla\mathbf{v}}_{\text{non-linear (convective) term}}\right)}_{\text{acceleration } (\mathrm{D}\mathbf{v}/\mathrm{D}t)} = \underbrace{-\nabla p}_{\text{pressure force}} + \underbrace{\mu\nabla^2\mathbf{v}}_{\text{viscous force}} + \underbrace{\mathbf{f}}_{\text{body force}}$$

$$\underbrace{\nabla\cdot\mathbf{v} = 0}_{\text{conservation of mass (incompressible)}}$$

A $1,000,000 Bounty Hunt

The Clay Millennium Problem

3D incompressible Navier-Stokes on $\mathbb{R}^3$

For smooth, divergence-free initial data with no forcing,
do solutions stay smooth forever, or can they break down?

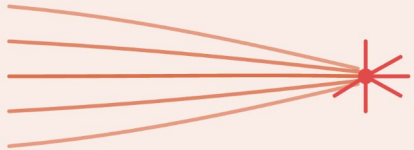
1 Existence & smoothness

A smooth, finite-energy solution
exists for every $t \geq 0$.

stays smooth for all time

2 Breakdown

Some smooth data give a solution
that blows up in finite time.



singularity forms at $t = T^*$

Differential Equations are the Language of Systems!

Physics: Schrödinger equation

$$i\hbar\frac{\partial\psi}{\partial t} = -\frac{\hbar^2}{2m}\nabla^2\psi + V\psi$$

the quantum state of a particle, evolving in time

Finance: Black–Scholes equation

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0$$

the fair price of an option as markets move

Medicine: reaction–diffusion

$$\frac{\partial u}{\partial t} = D\nabla^2 u + R(u)$$

tumour growth, nerve signals, drug transport

Materials: Cahn–Hilliard equation

$$\frac{\partial c}{\partial t} = M\nabla^2\left(f'(c) - \kappa\nabla^2 c\right)$$

how alloys separate into microstructures

The Need to Approximate

Analytical

A single exact formula v(x, t) satisfying the equations everywhere.

❌ **doesn’t exist for many settings**

Numerical

Approximate values of the solution, computed step by step on a computer.

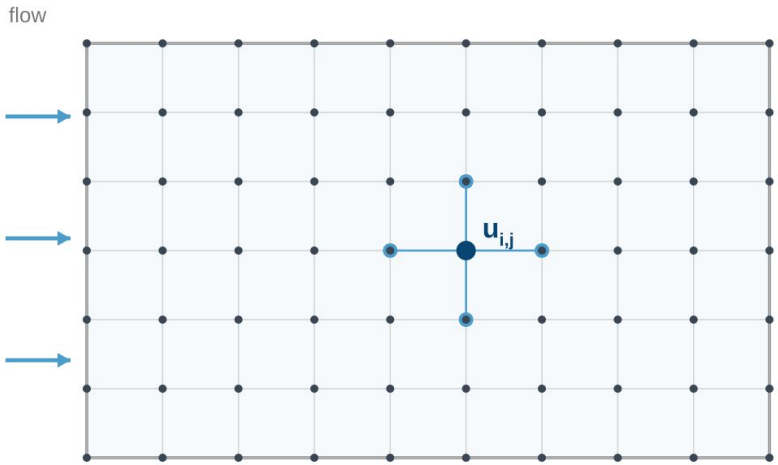
✅ **often possible, but expensive**

The idea: a mesh, solved at the nodes

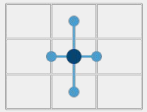
1. Cover the domain with a grid of nodes called a mesh.

2. The discrete solution is just the field value at each node, $\{ u_i \}$.

3. Approximate the equation at every node, resulting in a large system of algebraic equations to solve simultaneously.

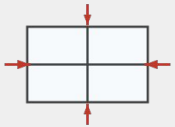


Common Numerical Techniques



Finite Differences (FDM)

Replace derivatives with differences between neighbouring nodes. Good for simple, structured grids.



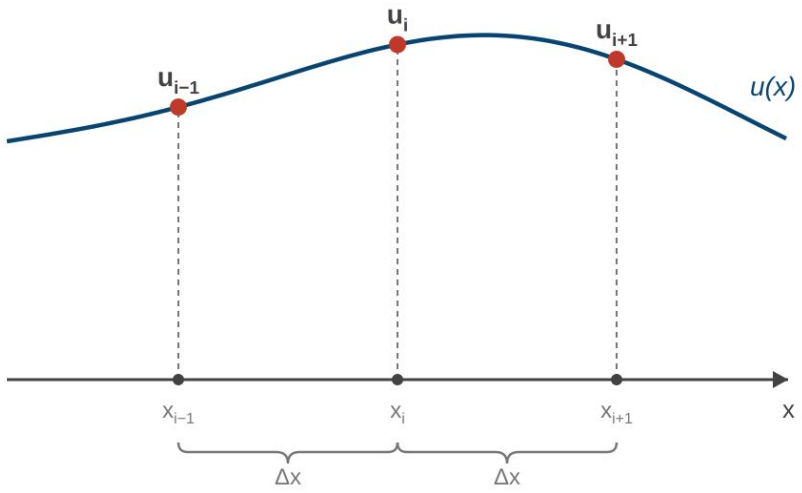
Finite Volumes (FVM)

Balance the flux in and out of each cell. The workhorse of computational fluid dynamics.

Spectral methods

Expand the solution in smooth global functions. Very accurate for smooth flows.

Example: Finite Difference Approximations



$$\left.\frac{\partial u}{\partial x}\right|_i \approx \frac{u_{i+1} - u_{i-1}}{2\,\Delta x}$$

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}$$

Each derivative becomes a simple combination of neighbouring node values. This is cheap to evaluate, and the error shrinks as $\Delta x \to 0$.

Example: Linear Advection

$$\frac{\partial u}{\partial t} + a\frac{\partial u}{\partial x} = 0$$

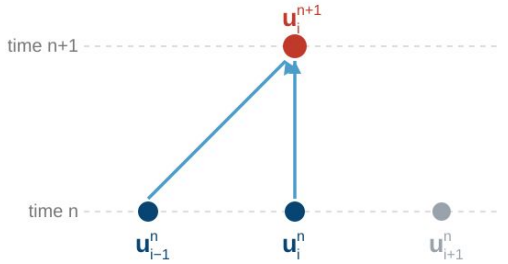
$$\underbrace{\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}}_{\text{forward in time}} \qquad \underbrace{\frac{\partial u}{\partial x} \approx \frac{u_i^n - u_{i-1}^n}{\Delta x}}_{\text{upwind in space}}$$

↓ substitute & rearrange

$$u_i^{n+1} = u_i^n - C\left(u_i^n - u_{i-1}^n\right), \qquad C = \frac{a\Delta t}{\Delta x}$$

The algorithm at every node and every step:

```
for n = 1, 2, 3, ...        # time steps
  for each node i:          # sweep the mesh
    u_new[i] = u[i] - C*(u[i] - u[i-1])
  u = u_new                 # advance in time
```



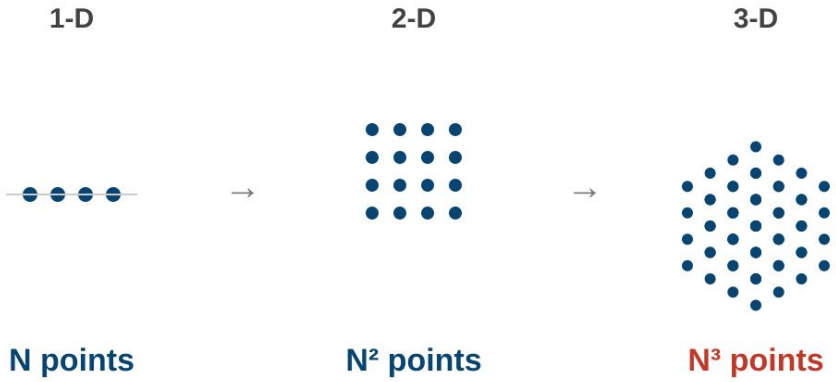
each new value uses the node and its **upwind neighbour**

Computational Complexity

stability (CFL): $\Delta t = \mathcal{O}(\Delta x^2)$

approximation error: $E = \mathcal{O}(\Delta x^2)$

solver complexity: $\mathcal{C} = \mathcal{O}\left(\left(\frac{1}{E}\right)^{\frac{d+2}{2}}\right)$



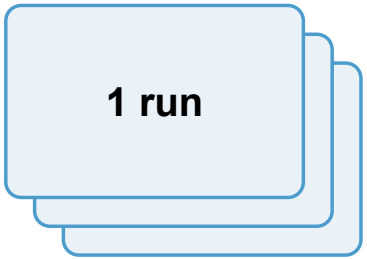
❌ **Computational complexity scales exponentially with problem dimension**

Multi-Query Problems

A single high-fidelity simulation already costs hours to days to weeks on a supercomputer.

But real work almost never needs just one:

Design sweeps	Optimisation	Uncertainty quantification	Real-time control



× **thousands of runs**

❌ **It simply does not scale**

Summary

1. **Observe** — a system of interest in the real world.
2. **Model** — write the governing equations that describe it.
3. **Solve** — analytically, to predict how it evolves, but rarely possible in closed form.
4. **Discretise** — approximate the solution numerically on a mesh… but the cost scales explosively.

A change in philosophy…

What if instead we could collect data from a system we're trying to model and learn its dynamics directly?

PART 2

AI for Science

What do we mean by AI for Science?

AI for Science applies machine learning to understand, model, and discover scientific systems.

Learn from scientific data

extract patterns from experiments or observations

Accelerate scientific models

approximate or enhance expensive simulations

Enable scientific discovery

search for new structures, materials, or hypotheses

Using an AI/ML model in practice

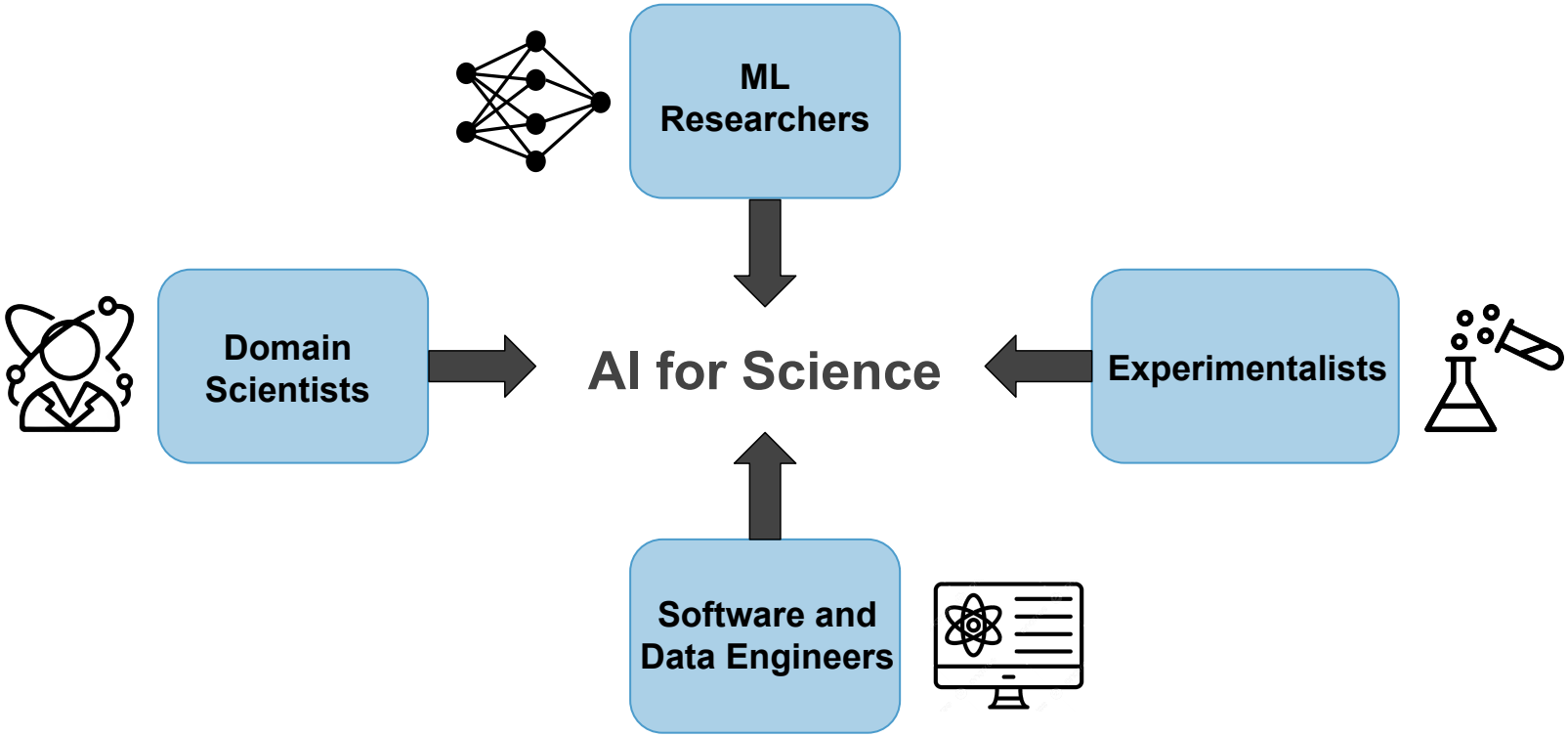
examples:

- image
- amino acid sequence
- stimulus injection
- weather state

examples:

- classify object
- predict protein structure
- predict membrane potential
- predict future atmosphere

Who is involved in AI for Science



What’s changed relative to the classical workflow?

Classical Scientific Computing

Define governing equations

Discretise and **solve** equations

Analyse outputs

AI-Enabled Workflow

Generate and/or collect observational data

Train a surrogate AI model

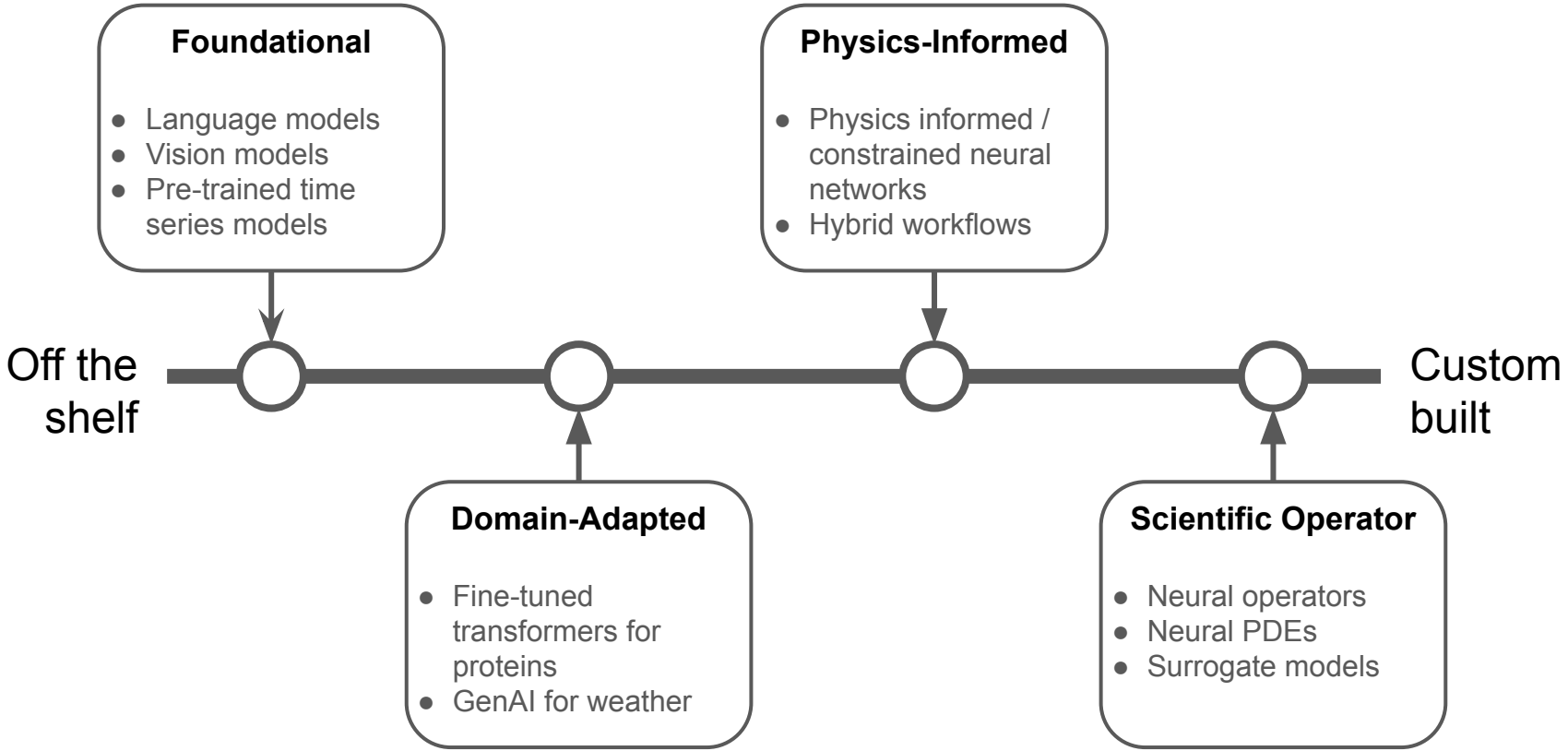
Validate model predictions on experimental recordings

Why surrogate modelling matters

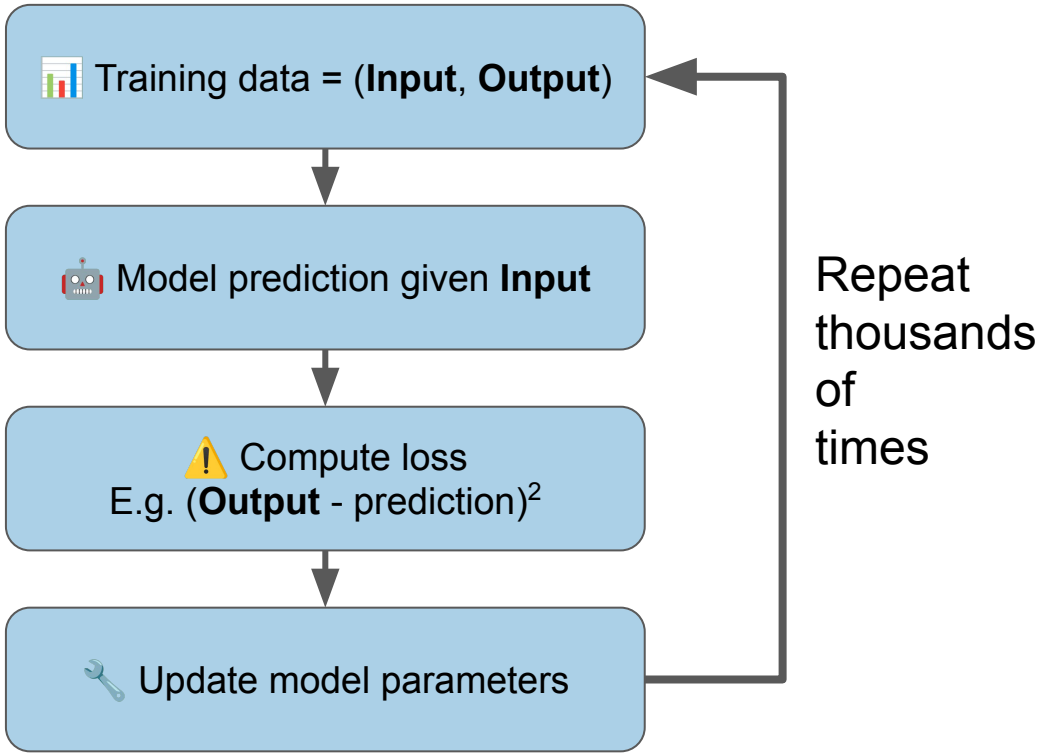
One simulation may be feasible. Ten thousand often is not.

Classical Scientific Computing	**High-Accuracy Simulator**	**Slow Repeated Queries**	**Too Expensive At Scale**
AI-Enabled Workflow	**Learned Surrogate**	**Fast Repeated Queries**	**Feasible At Scale**

What kinds of models are used?



Our Old friend: The Standard AI/ML Training Workflow

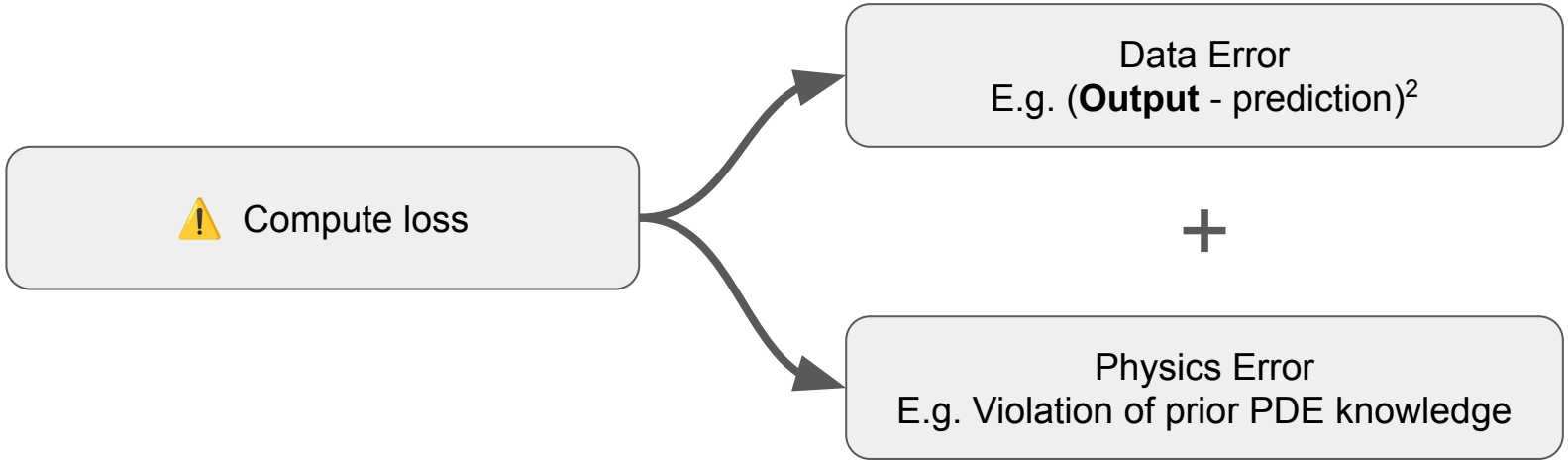


PART 3

Physics Informed Machine Learning

Physics-Informed Workflow

Same as the standard ML Workflow, except:



Problem setup: a general PDE

Consider a general PDE with its initial and boundary conditions:

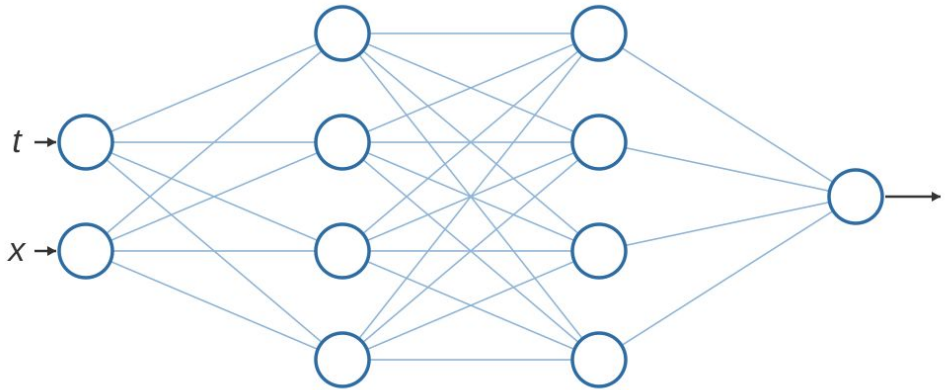
$$\mathcal{D}[\,u(t,x)\,] = f(t,x)$$

$$\mathcal{B}_k[\,u(t,x)\,] = g_k(t,x)$$

- the differential operator defines the PDE,
- the boundary operators fix the initial & boundary data
- u(t, x) is the solution we want to find.

Learn the physics with a neural network

Train a neural network to approximate the PDE solution:



$$\mathrm{NN}(t, x; \theta) \approx u(t, x)$$

Physics-informed loss

Idea: Guide training with a loss that adds the physics as soft constraints:

$$\mathcal{L}(\theta) = \mathcal{L}_b(\theta) + \mathcal{L}_p(\theta)$$

Boundary loss

$$\mathcal{L}_b(\theta) = \sum_k \frac{\lambda_k}{N_{b_k}} \sum_j \Big(\mathcal{B}_k \left[\mathrm{NN}(x_{kj}; \theta) \right] - g_k(x_{kj}) \Big)^2$$

matches the initial & boundary conditions

Physics loss

$$\mathcal{L}_p(\theta) = \frac{1}{N_p} \sum_i \Big(\mathcal{D}[\mathrm{NN}(x_i; \theta)] - f(x_i) \Big)^2$$

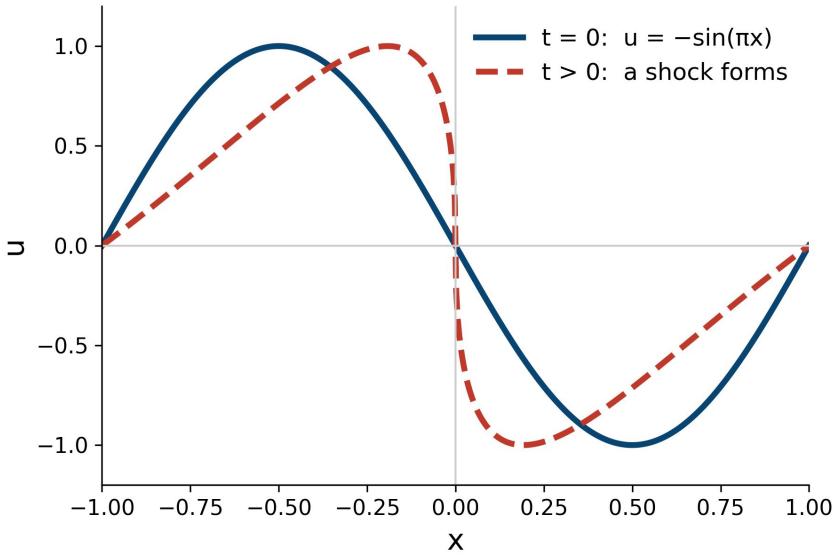
penalises any violation of the underlying equation

Example: 1D viscous Burgers' equation

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} - \nu\frac{\partial^2 u}{\partial x^2} = 0$$

$$u(x,0) = -\sin(\pi x)$$

$$u(-1,t) = u(1,t) = 0$$



The boundary loss

One term for the initial condition and two boundary conditions:

$$\mathcal{L}_b(\theta) = \underbrace{\frac{\lambda_1}{N_{b_1}} \sum_j \left(\mathrm{NN}(x_j, 0; \theta) + \sin(\pi x_j)\right)^2}_{\text{initial condition}} + \underbrace{\frac{\lambda_2}{N_{b_2}} \sum_k \left(\mathrm{NN}(-1, t_k; \theta) - 0\right)^2}_{\text{left boundary}} + \underbrace{\frac{\lambda_3}{N_{b_3}} \sum_l \left(\mathrm{NN}(1, t_l; \theta) - 0\right)^2}_{\text{right boundary}}$$

The physics loss

Penalise the PDE residual at points sampled inside the domain:

$$\mathcal{L}_p(\theta) = \frac{1}{N_p} \sum_i \left[\left(\frac{\partial \mathrm{NN}}{\partial t} + \mathrm{NN} \frac{\partial \mathrm{NN}}{\partial x} - \nu \frac{\partial^2 \mathrm{NN}}{\partial x^2} \right) (x_i, t_i; \theta) - 0 \right]^2$$

The derivative of the neural network can be computed exactly using autodifferentiation!

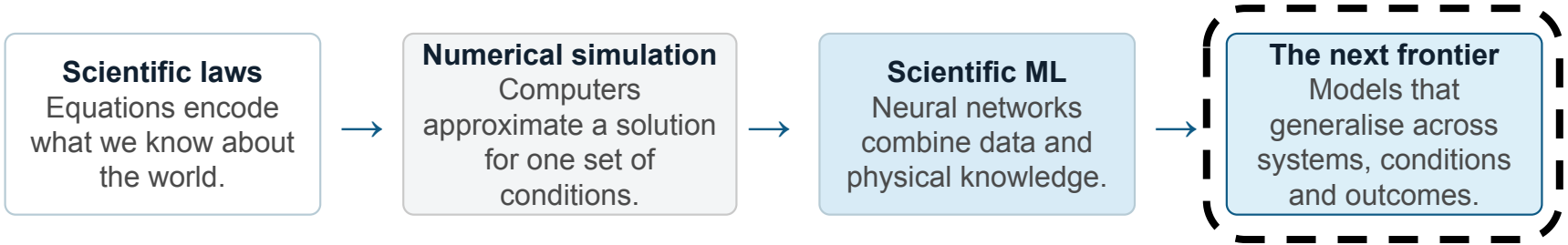
Computing the loss function via autodifferentiation

```
# boundary loss  —  evaluate the network at the IC / BC points

ic    = NN(x0, 0, theta) + torch.sin(pi*x0)
left  = NN(-1, tk, theta) - 0
right = NN( 1, tl, theta) - 0
boundary_loss = torch.mean(ic**2) + torch.mean(left**2) + torch.mean(right**2)


# physics loss  —  derivatives of the network via autodifferentiation

t.requires_grad_(True);  x.requires_grad_(True)
u = NN(t, x, theta)
dudt   = torch.autograd.grad(u, t, torch.ones_like(u), create_graph=True)[0]
dudx   = torch.autograd.grad(u, x, torch.ones_like(u), create_graph=True)[0]
d2udx2 = torch.autograd.grad(dudx, x, torch.ones_like(u), create_graph=True)[0]
physics_loss = torch.mean((dudt + u*dudx - eta*d2udx2)**2)


loss = boundary_loss + physics_loss
```

PART 4

Outlook

From Solving Equations to Learning Scientific Systems



AI for Science does not replace scientific knowledge.
It combines equations, data and computation to make new kinds of scientific modelling possible.

Next time…

From computing one solution to learning reusable scientific models

Can neural networks can learn maps between entire families of scientific problems and their solutions, and even generate new scientific candidates?

Neural operators **Generative models** **Foundation models**